
Variable-Length 2D Arrays and Row-Major Order

SUBMITTED BY

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

Question 1

Problem Statement

Write a C program to:

1. Read the number of rows and columns from the user. Ensure that both values are positive. If not, repeat to get the desired input(s).
2. Declare a variable-length 2D integer array using the given dimensions.
3. Read the array elements column by column using nested loops, with the outer loop iterating over columns and the inner loop over rows.
4. Print the elements in row-major order using a single loop and a linear index.

Algorithm

1. Start the program and declare the variables rows and cols.
2. Read rows using a do-while loop; repeat while rows is not positive.
3. Read cols using a do-while loop; repeat while cols is not positive.
4. Declare the variable-length array `arr[rows][cols]`.
5. Use an outer loop `j` from 0 to `cols-1` and an inner loop `i` from 0 to `rows-1`, and read each value into `arr[i][j]`.
6. Take a pointer `p` to the first element, `&arr[0][0]`.
7. Run a single loop `k` from 0 to `(rows * cols) - 1` and print `p[k]`.
8. Stop the program.

Program

```
#include <stdio.h>

int main(void)
{
    int rows, cols;

    /* (a) Read rows and columns; repeat until both are positive */
    do {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        if (rows <= 0)
            printf("Rows must be positive. Please try again.\n");
    } while (rows <= 0);

    do {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
        if (cols <= 0)
            printf("Columns must be positive. Please try again.\n");
    } while (cols <= 0);

    /* (b) Variable-length 2D array */
    int arr[rows][cols];

    /* (c) Read elements column by column */
    printf("\nEnter the elements column by column:\n");
    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows; i++) {
            printf("Element [%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }

    /* (d) Print in row-major order using a single loop and linear index */
    printf("\nElements in row-major order:\n");
    int *p = &arr[0][0];
    for (int k = 0; k < rows * cols; k++) {
        printf("%d ", p[k]);
    }
    printf("\n");

    return 0;
}
```

Sample Output

```
Enter number of rows: -2
Rows must be positive. Please try again.
Enter number of rows: 2
Enter number of columns: 0
Columns must be positive. Please try again.
Enter number of columns: 3

Enter the elements column by column:
Element [0][0]: 1
Element [1][0]: 2
Element [0][1]: 3
Element [1][1]: 4
Element [0][2]: 5
Element [1][2]: 6

Elements in row-major order:
1 3 5 2 4 6
```

Explanation

- The do-while loops keep asking for a value until it is greater than zero, so invalid dimensions such as -2 or 0 are rejected.
- `int arr[rows][cols]` is a variable-length array (VLA), whose size is decided at run time from the user's input.
- Because the outer loop runs over columns and the inner loop over rows, the values are entered down each column, so the input order is 1, 2 | 3, 4 | 5, 6.
- A 2D array is stored in memory row by row. Using the pointer `p = &arr[0][0]`, the single index `k` walks through memory in row-major order, so the values are printed as 1 3 5 2 4 6.

Question 2

Problem Statement

Write a C program that reads elements into a variable-length 2D array, copies them into a 1D array in row-major order, and displays both arrays.

Algorithm

1. Start the program and declare the variables rows and cols.
2. Read rows and cols, repeating until both are positive.
3. Declare the 2D array `arr[rows][cols]` and the 1D array `linear[rows * cols]`.
4. Read the elements of `arr` using nested loops (row by row).
5. Set `k = 0`. For every `i` and `j`, copy `arr[i][j]` into `linear[k]` and increase `k` by 1.
6. Display the 2D array in matrix form using nested loops.
7. Display the 1D array using a single loop.
8. Stop the program.

Program

```
#include <stdio.h>

int main(void)
{
    int rows, cols;

    do {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        if (rows <= 0)
            printf("Rows must be positive. Please try again.\n");
    } while (rows <= 0);

    do {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
        if (cols <= 0)
            printf("Columns must be positive. Please try again.\n");
    } while (cols <= 0);

    int arr[rows][cols];          /* variable-length 2D array */
    int linear[rows * cols];      /* 1D array */

    /* Read elements of the 2D array */
    printf("\nEnter the elements row by row:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("Element [%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }

    /* Copy into 1D array in row-major order */
    int k = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            linear[k++] = arr[i][j];
        }
    }

    /* Display the 2D array */
    printf("\n2D array:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%4d", arr[i][j]);
        }
        printf("\n");
    }

    /* Display the 1D array */
    printf("\n1D array (row-major order):\n");
    for (int i = 0; i < rows * cols; i++) {
        printf("%d ", linear[i]);
    }
    printf("\n");

    return 0;
}
```

Sample Output

```
Enter number of rows: 2
Enter number of columns: 3

Enter the elements row by row:
Element [0][0]: 1
Element [0][1]: 2
Element [0][2]: 3
Element [1][0]: 4
Element [1][1]: 5
Element [1][2]: 6

2D array:
  1  2  3
  4  5  6

1D array (row-major order):
1 2 3 4 5 6
```

Explanation

- Both arrays are variable-length arrays, so their sizes come from the values entered by the user at run time.
- In row-major order, all elements of row 0 come first, then row 1, and so on. The counter *k* increases by one after every copy, which places each element in its correct position in the 1D array.
- The 2D array is displayed as a matrix, and the 1D array is displayed on a single line, so both can be compared easily.